

src\case_studies\F_vtol\pid_controller.py

Generated: 2026-02-26 12:33:40

pid_controller.py

src\case_studies\F_vtol\pid_controller.py

```
# 3rd-party
import numpy as np

# Local (controlbook)
from .. import common
from .. import params as P
from ..control.pd import PD
from ..control.pid import PID
from ..control.dirty_derivative_filter import DirtyDerivativeFilter
from ..control import utils_design

class VTOLControllerPID(common.ControllerBase):
    def __init__(self):
        # tuning parameters
        tr_h = 3.0
        zeta_h = 0.95
        tr_theta = 0.6
        zeta_theta = 0.9
        M = 10.0 # time scale separation between inner and outer loop
        tr_z = tr_theta * M
        zeta_z = 0.9

        ki_h = 1.2
        ki_z = -1e-5 # works better with 0

        # system parameters
        a1_lon, a0_lon = P.tf_lon_den[-2:]
        a1_lat_inner, a0_lat_inner = P.tf_lat_inner_den[-2:]
        a1_lat_outer, a0_lat_outer = P.tf_lat_outer_den[-2:]
        b0_lon = P.tf_lon_num[-1]
        b0_lat_inner = P.tf_lat_inner_num[-1]
        b0_lat_outer = P.tf_lat_outer_num[-1]
        a0_lat_inner = P.tf_lat_inner_den[-1]

        # longitudinal (altitude) gains
        alpha1_lon, alpha0_lon = utils_design.get_des_CE(tr_h, zeta_h)
        kp_h = (alpha0_lon - a0_lon) / b0_lon
        kd_h = (alpha1_lon - a1_lon) / b0_lon
        self.h_pid = PID(kp_h, ki_h, kd_h, P.ts, anti_windup_vel_tol=0.1)
        print(f"longitudinal gains: {kp_h = :.3f}, {ki_h = :.2f}, {kd_h = :.3f}")

        ### Lateral (side-to-side) gains
        # inner loop (theta)
        alpha1_lat_inner, alpha0_lat_inner = utils_design.get_des_CE(
            tr_theta, zeta_theta
        )
        kp_theta = (alpha0_lat_inner - a0_lat_inner) / b0_lat_inner
        kd_theta = (alpha1_lat_inner - a1_lat_inner) / b0_lat_inner
        self.theta_pd = PD(kp_theta, kd_theta)
        print(f"lateral inner loop gains: {kp_theta = :.3f}, {kd_theta = :.3f}")

        # DC gain of inner loop
```

```

DC_gain = (b0_lat_inner * kp_theta) / (a0_lat_inner + b0_lat_inner * kp_theta)
print(f"DC_gain = {DC_gain}") # a0 is 0, so DC_gain is 1

# outer loop (z)
effective_b0_outer = b0_lat_outer * DC_gain
alpha1_lat_outer, alpha0_lat_outer = utils_design.get_des_CE(tr_z, zeta_z)
kp_z = (alpha0_lat_outer - a0_lat_outer) / effective_b0_outer
kd_z = (alpha1_lat_outer - a1_lat_outer) / effective_b0_outer
self.z_pid = PID(kp_z, ki_z, kd_z, P.ts, anti_windup_vel_tol=0.1)
print(f"lateral outer loop gains: {kp_z = .3f}, {ki_z = .5f}, {kd_z = .3f}")

self.u_FT_eq = P.u_FT_eq
self.theta_max = np.radians(10) # TODO: use or not?

# dirty derivative filters (derivatives calculated outside PD/PID classes)
self.z_dot_filter = DirtyDerivativeFilter(P.ts, sigma=0.05, x0=P.z0)
self.h_dot_filter = DirtyDerivativeFilter(P.ts, sigma=0.05, x0=P.h0)
self.theta_dot_filter = DirtyDerivativeFilter(P.ts, sigma=0.05)

def update_with_state(self, r, x):
    raise NotImplementedError("This controller only uses measurement feedback.")

def update_with_measurement(self, r, y):
    z, h, theta = y
    z_ref, h_ref = r[:2]

    # calculate derivatives externally using dirty derivative filters
    z_dot = self.z_dot_filter.update(z)
    h_dot = self.h_dot_filter.update(h)
    theta_dot = self.theta_dot_filter.update(theta)

    # altitude (modified) PID with explicit derivative
    F_tilde = self.h_pid.update_modified(h_ref, h, h_dot)

    # lateral outer loop (modified) PID with explicit derivative
    theta_ref = self.z_pid.update_modified(z_ref, z, z_dot)

    # TODO: decide if we want to saturate the angle
    # theta_ref = np.clip(theta_ref, -self.theta_max, self.theta_max)
    r[2] = theta_ref # if you want to visualize the "reference" angle

    # lateral inner loop (modified) PD with explicit derivative
    tau_tilde = self.theta_pd.update_modified(theta_ref, theta, theta_dot)

    # combine lateral and longitudinal control
    u_FT_tilde = np.array([F_tilde, tau_tilde])
    u_FT = u_FT_tilde + self.u_FT_eq

    # convert force-torque to motor forces
    u_unsat = P.mixer @ u_FT
    u = self.saturate(u_unsat, u_max=P.fr_max, u_min=0.0)

    # doesn't work as well as velocity-based anti-windup
    # u_FT_sat = P.unmixer @ u
    # self.h_pid.anti_windup_from_saturation(u_FT_sat[0], u_FT[0])
    xhat = np.array([z, h, theta, z_dot, h_dot, theta_dot])

    return u, xhat.astype(np.float64)

```